

Методические рекомендации

СОЗДАНИЕ SQL-ЗАПРОСОВ В РЕЛЯЦИОННЫХ СУБД

1. Реляционные языки программирования и манипулирования данными

Реляционные языки оперируют с данными как с множествами, применяя к ним операции теории множеств. На входе реляционного оператора – множество записей одной или нескольких таблиц, на выходе – множество записей новой таблицы. Выделяют следующие разновидности языков.

dBASE-подобные языки обеспечивают создание интерфейса пользователя, выполнение основных типовых операций, таких как добавление и редактирование записей, удаление, поиск, копирование записей и т.п. Они обладают выраженной процедурностью обработки, когда явно указывается последовательность действий, приводящих к конечному результату.

Графические реляционные языки ориентированы на конечных пользователей. Типичным представителем такого языка является *QBE (Query By Example)*, реализованного в среде электронных таблиц, в ряде СУБД (в частности в *Access*), в пакете *Microsoft Query*. Функции языка доступны в формах различного рода меню, диалоговых сценариях или заполняемых пользователем таблицах. По таким входным данным интерфейс формирует адекватные синтаксические конструкции и передает их на исполнение.

SQL-подобные (Structured Query Language) языки запросов реализуются в большинстве многопользовательских и распределённых СУБД. Язык *SQL* стал стандартом языков запросов для работы с реляционными базами.

SQL предназначен для выполнения операций над таблицами (создание, удаление, изменение структуры), данными таблиц (выборка, изменение, добавление и удаление), а также некоторых сопутствующих операций. *SQL* является не процедурным языком и не содержит операторов управления и организации подпрограмм. В связи с этим *SQL* обычно погружен в среду встроенного языка программирования СУБД (например, языка СУБД *Visual FoxPro*) или даже процедурного языка типа *C++* или *Pascal*.

Основные операторы усеченного подмножества *SQL*:

CREATE TABLE, DROP TABLE - создание, удаление таблицы;

CREATE INDEX, DROP INDEX - создание, удаление индекса;

ALTER TABLE - изменение структуры таблицы;

SELECT, UPDATE, INSERT, DELETE - выборка, изменение, вставка и удаление записей.

К дополнительным операторам *SQL* относят:

CREATE DATABASE, SHOW DATABASE, START DATABASE, STOP DATABASE, DROP DATABASE - создание, просмотр, активизация, закрытие, удаление базы данных (БД);

CREATE VIEW, DROP VIEW - создание, удаление выборки (представления);

CREATE SYNONYM - создание синонима;

GRANT, REVOKE - назначение, удаление привилегии для работы с выборками и таблицами.

Для удовлетворения сложных информационных потребностей пользователи «общаются» с БД с помощью запросов. Запрос представляет собой спецификацию (предписание) на специальном языке (базы данных) для обработки данных. В реляционных СУБД запросы к БД выражаются с помощью *SQL*-инструкции *SELECT*. В упрощенном виде оператор *SELECT* имеет следующий формат:

SELECT [ALL-DISTINCT] <СписокДанных - ВыбираемыхПолей>

FROM <СписокТаблиц - источник данных> [INTO ИмяТаблицы получателя данных] [WHERE <условие выборки>] [GROUP BY Условие группировки <Имя столбца>] [<имя столбца>][HAVING <Условие поиска>] [ORDER BY Условие упорядочения выводимых данных <спецификация сортировки>][<спецификация сортировки>] [TO FILE ИмяФайла (TO PRINTER – направление вывода данных)]

Оператор *SELECT* позволяет выполнять выборку и вычисления над данными одной или нескольких таблиц. Результатом является таблица, которая может иметь (*ALL*) или не иметь

(*DISTINCT*) повторяющиеся строки.

В списке данных можно задавать имена столбцов и выражения над ними, к примеру, арифметические. Если записи отбираются из нескольких таблиц, то используют составные имена <имя таблицы>.<имя столбца>.

С точки зрения решаемых информационных задач и формы результатов исполнения запросов их можно разделить на три группы: запросы на выборку данных; запросы на изменение данных; управляющие запросы.

Вход в режим SQL. Формирование запросов в СУБД может осуществляться в специальном редакторе (командный режим) или через диалоговые средства (конструкторы) и пошаговые мастера. Сформированный запрос в виде SQL-инструкции сохраняется в файле БД и затем запускаться на выполнение.

В современных СУБД с интерактивным интерфейсом можно создавать запросы, не применяя SQL. Однако его применение позволяет расширить возможности использования СУБД. К примеру, в Access можно перейти из окна конструктора запросов в окно с оператором SQL. Для этого необходимо, находясь в режиме «Создание запроса в режиме Конструктор», нажать левую кнопку мыши и в появившемся контекстном меню выбрать пункт «Режим SQL». В появившемся окне следует набрать инструкцию SQL. При закрытии окна следует выполнить запоминание запроса с уникальным именем.

Запросы на выборку являются наиболее часто применяемыми запросами. Обычно они реализуются SQL-инструкцией *SELECT* с предложением *FROM*.

Результатом исполнения запроса на выборку является *набор данных*, который представляет *временную таблицу* со структурой, определяемой параметрами запроса и полей таблиц, из которых выбираются данные. В отличие от режимов поиска и фильтрации запросами на выборку данные выбираются из закрытых таблиц.

Результаты запросов на выборку помещаются в специальную временную таблицу, размещаемую на период исполнения запроса в оперативной памяти. В этом смысле запрос в реляционных СУБД *тождественен* просто таблице данных, «открытие» которой осуществляется в результате выполнения запроса. Из этого следует возможность исполнения запросов над результатами исполнения других запросов, что облегчает построение запросов при решении сложных задач.

Наборы данных, формируемые запросами на выборку, являются *динамическими*. Это означает, что с результатом исполнения запроса можно производить все те же операции, что и с данными в режиме открытой таблицы.

Запросы на выборку классифицируются по двум критериям - по формированию условий выборки и по схеме отбора данных.

По формированию условий выборки запросы можно подразделить на запросы со статическими (неизменяемыми) условиями отбора, запросы с параметрами и запросы с подчиненными запросами.

По схеме отбора данных запросы на выборку подразделяются на запросы на выборку данных из одной таблицы, запросы на выборку данных в один набор из нескольких таблиц и запросы на объединение данных.

2. Запросы на выборку данных из одной таблицы

Запросы на выборку данных из одной таблицы по смыслу и назначению сходны с фильтрацией данных в открытой таблице. Различие заключается лишь в форме представления результата. В частности, запросом на выборку можно отображать не просто подмножество записей исходной таблицы, но и подмножество ее полей исходной таблицы, а над результатным набором данных, как уже отмечалось, можно исполнить другой запрос.

Различают запросы на выборку всех записей с произвольным набором полей и запросы на выборку подмножества записей.

На рисунке 2.1. приведен пример запроса, формирующего список сотрудников организации из таблицы «Сотрудники», но с сокращенным набором полей.

Сотрудники						
Таб_№	Фамилия	Имя	Отчество	Должность	Подразделение	Телефон
1	Иванов	Иван	Иванович	Начальник	Отдел№0	100001
2	Петров	Петр	Петрович	Начальник	Отдел№1	111101
3	Сидоров	Сидор	Сидорович	Инженер	Отдел№1	111102
4	Егоров	Егор	Егорович	Начальник	Отдел№3	111001
5	Кузьмина	Ольга	Игоревна	Секретарь	Отдел№3	111001

Запрос на выборку всех записей с произвольным набором полей
SELECT [Сотрудники].[Таб_№], [Сотрудники].[Фамилия], [Сотрудники].[Имя]
FROM Сотрудники;

Список сотрудников

Таб_№	Фамилия	Имя
1	Иванов	Иван
2	Петров	Петр
3	Сидоров	Сидор
4	Егоров	Егор
5	Кузьмина	Ольга

Рисунок 2.1. - Пример запроса на выборку всех записей по группе полей

В запросах на отбор подмножества записей в SQL-инструкции *SELECT* через предложение *WHERE* помещается выражение, определяющее условие отбора данных. На рисунке 2.2. приведен пример запроса на отбор подмножества записей из таблицы «Сотрудники» для формирования списка работников инженерно-технического и экономического профиля.

Сотрудники				
Фамилия	Должность	Кабинет	Телефон	Ученая_степень
Егорова	Секретарь	101a	1101	
Иванов	Начальник	101	1101	д.т.н.
Иванова	Секретарь	106	1105	
Петров	Начальник	110	1110	к.т.н.
Петрова	Секретарь	106	1105	
Сидоров	Экономист	110	1110	к.э.н.
Сидорова	Секретарь	106	1105	
Фетисов	Инженер	105	1105	

Запрос на выборку
SELECT Сотрудники.Фамилия, Сотрудники.Должность
FROM Сотрудники
WHERE ((Сотрудники.Должность)="Инженер" Or (Сотрудники.Должность)="Экономист");

Синие воротнички

Фамилия	Должность
Петров	Инженер
Сидоров	Экономист
Фетисов	Инженер

Рисунок 2.2. - Пример запроса на выборку подмножества записей

В запросах на выборку данных широко применяются *предикаты отбора ALL, DISTINCT, DISTINCTROW и TOP n*.

Предикат *ALL* используется по умолчанию и устанавливает вывод в набор всех записей, формируемых по условию отбора в предложении *WHERE*, и в большинстве случаев в инструкции *SELECT* опускается.

Предикат *DISTINCT* используется для исключения в наборе отбираемых данных тех записей, значения которых по определенному полю повторяются, т. е. уже раз вошли в набор. На рисунке 2.3. приведен пример запроса, отбирающего из таблицы «Сотрудники» данные по полю «Должность» без предиката отбора (т. е. с предикатом *ALL*) и с предикатом *DISTINCT*. Использование предиката *DISTINCT* позволяет сформировать простой список должностей без повторов.

Предикат *DISTINCTROW* имеет аналогичное предикату *DISTINCT* назначение для исключения из набора тех записей, значения которых повторяются по всем полям, включенным в набор данных.

Предикат *TOP n* обеспечивает включение в набор данных первых *n* записей, сформированных по условию отбора. Пример запроса с предикатом *TOP n* на рисунке 2.3.

В запросах на выборку помимо предложений *FROM* и *WHERE* используются предложения *GROUP BY, HAVING* и *ORDER BY* для дополнительной обработки отбираемых записей.

Сотрудники				
Таб. №	Фамилия	Имя	Отчество	Должность
1	Белых	Б.	Б.	Генеральный директор
3	Иванова	И.	П.	Секретарь-референт
4	Сидоров	С.	С.	Экономист
6	Петров	П.	П.	Начальник отдела
9	Тутова	О.	Н.	Секретарь-референт
10	Попова	С.	О.	Начальник группы
11	Васильева	В.	В.	Бухгалтер
15	Егорова	Е.	Е.	Инспектор
17	Надеждин	С.	С.	Начальник отдела
20	Сухов	С.	С.	Инженер

Список должностей с повторами	Список должностей без повторов	Список первых пяти должностей
SELECT [Сотрудник].[Должность] FROM Сотрудник;	SELECT DISTINCT [Сотрудник].[Должность] FROM Сотрудник;	SELECT TOP 5 Сотрудник.Должность FROM Сотрудник;

Должность	Должность	Должность
Генеральный директор	Генеральный директор	Генеральный директор
Секретарь-референт	Секретарь-референт	Секретарь-референт
Экономист	Экономист	Экономист
Начальник отдела	Начальник отдела	Начальник отдела
Секретарь-референт	Начальник группы	Секретарь-референт
Начальник группы	Бухгалтер	
Бухгалтер	Инспектор	
Инспектор	Инженер	
Начальник отдела		
Инженер		

Рисунок 2.3. - Пример запросов с предикатами *ALL, DISTINCT* и *TOP n*

Предложение *GROUP BY* объединяет (группирует) записи с одинаковыми значениями определенных полей в одну запись. Предложение *HAVING* выполняет функцию предложения *WHERE*, позволяя задавать дополнительные условия для отбора сгруппированных предложением *GROUP BY* записей.

Предложение *ORDER BY* обеспечивает сортировку отобранных записей в зависимости от способа *ASC* (по возрастанию) или *DESC* (по убыванию).

На рисунке 2.4. приведен пример запроса, формирующего в порядке убывания список сгруппированных по полям «Категория» и «Профиль» записей из таблицы «Подразделения» при условии отбора подразделений с категорий выше третьей и отбора сгруппированных записей при условии основного профиля подразделений.

Подразделения			
Усл_наименование	Категория	Профиль	Телефон
Бухгалтерия	Третья	Обеспечивающий	777
Отдел кадров	Вторая	Вспомогательный	333
Отдел режима	Третья	Вспомогательный	110
Отдел сбыта	Вторая	Основной	666
Отдел снабжения	Вторая	Основной	444
Планово-экономический отдел	Вторая	Основной	888
Производственный отдел	Первая	Основной	555
Руководство	Первая	Основной	111
Секретариат	Третья	Обеспечивающий	222

Категории подразделений с профилем «Основной»

```

SELECT Подразделения.Категория, Подразделения.Профиль
FROM Подразделения
WHERE ((Подразделения.Категория)<>"Третья")
GROUP BY Подразделения.Категория, Подразделения.Профиль
HAVING ((Подразделения.Профиль)="Основной")
ORDER BY Подразделения.Категория DESC;

```

Категория	Профиль
Первая	Основной
Вторая	Основной

Рисунок 2.4. - Пример запроса с предложениями *GROUP BY*, *HAVING* и *ORDER BY*

Специфику имеет отбор записей с «пустыми» значениями определенных полей. В реляционных СУБД и языке SQL «пустых», т. е. неопределенных, значений полей не бывает. Иначе говоря, значением числового поля может быть число, равное «0», а значением других типов полей (текстовые, дата) может быть нулевое значение - «Null».

Отбор записей с пустыми значениями применяется тогда, когда нужно отдельно сформировать и проанализировать набор данных с записями, содержащими нулевые значения для числовых полей, или не имеющие, в силу каких-либо причин, определенного значения для других типов полей.

Запрос по поиску пустых значений			
SELECT Сотрудники.Фамилия, Сотрудники.Должность, Сотрудники.Кабинет, Сотрудники.Телефон FROM Сотрудники WHERE ((Сотрудники.Ученая_степень) Is Null);			
Сотрудники, не имеющие ученых степеней			
Фамилия	Должность	Кабинет	Телефон
Фетисов	Инженер	105	1105
Егорова	Секретарь	101-а	1101
Иванова	Секретарь	106	1105
Петрова	Секретарь	106	1105
Сидорова	Секретарь	106	1105

Рисунок 2.5. - Запрос по поиску данных с пустыми значениями определенного поля

На рисунке 2.5. представлен пример запроса, отбирающего данные из таблицы «Сотрудники» (рисунок 2.2.) с «пустыми» значениями по полю «Ученая степень», иначе говоря, формирующий список сотрудников, не имеющих ученых степеней.

Лабораторная работа № 1. Создайте таблицу «Родственники» с полями «Родство», «Имя», «Возраст», «Работа», «Должность» и сформируйте запросы:

1. Запрос на два произвольных поля.
2. Запрос на выборку в поле «Должность» подмножества записей (пенсионер, ребенок);
3. Запрос на список должностей с повторами, без повторов и первые три.
4. Запрос на список имен дедушек и бабушек с указанием возраста по убыванию.
5. Запрос на список не работающих родственников.

3. Запросы на выборку данных из нескольких таблиц

Запросы данного типа подразделяются на три группы: запросы на сочетание данных, запросы на соединение данных и запросы на соединение данных с указанием критериев отбора.

Запросы на сочетание направлены на формирование полного набора сочетаний записей исходных таблиц и строятся на основе SQL-инструкции *SELECT* и предложения *FROM* с простым перечислением отбираемых полей и их таблиц.

На рисунке 3.1. приведен запрос на выборку сочетания данных из таблицы «Подразделения» и таблицы «Мероприятия». Формирование такого запроса может быть обусловлено потребностями полного набора сочетаний данных по подразделениям и по мероприятиям.

Подразделения			Мероприятия	
Наименование	Руководитель	Кол_сотр	Вид	Дата
Отдел сбыта	Петров	43	Квартальный отчет	01.04.98
Отдел снабжения	Иванов	23	Квартальный отчет	01.07.98
Производств. отдел	Сидоров	3	Полугодовой отчет	10.07.98

Запрос на сочетание данных

SELECT Подразделения.*, Мероприятия.*
FROM Мероприятия, Подразделения;

План-график мероприятий

Наименование	Руководитель	Кол_сотр	Вид	Дата
Отдел снабжения	Иванов	23	Квартальный отчет	01.04.98
Отдел снабжения	Иванов	23	Квартальный отчет	01.07.98
Отдел снабжения	Иванов	23	Полугодовой отчет	10.07.98
Отдел сбыта	Петров	43	Квартальный отчет	01.04.98
Отдел сбыта	Петров	43	Квартальный отчет	01.07.98
Отдел сбыта	Петров	43	Полугодовой отчет	10.07.98
Производств. отдел	Сидоров	3	Квартальный отчет	01.04.98
Производств. отдел	Сидоров	3	Квартальный отчет	01.07.98
Производств. отдел	Сидоров	3	Полугодовой отчет	10.07.98

Рисунок. 3.1. - Пример реализации запроса на сочетание данных из двух таблиц

Запрос на соединение данных предполагает сведение данных соответствующих полей из различных таблиц в единую таблицу с сохранением каждого поля. Их построение осуществляется на основе SQL-инструкции *SELECT* с перечислением отбираемых полей и их таблиц. Предложения *FROM* строятся по структуре (FROM <Таблица1> INNER JOIN <Таблица2> ON <Таблица1.Код>=<Таблица2.Код>) с совпадением кодов обеих таблиц.

Запрос на соединение данных с указанием критериев отбора предполагает сведение выбранных данных соответствующих полей из различных таблиц в единую таблицу на основе заданного параметра. Их построение осуществляется аналогично запросу на соединение данных с добавлением указания критерия отбора. WHERE (((Таблица.Поле)='Параметр'));

Лабораторная работа № 2. Дополнительно к уже имеющейся создайте таблицу «Родственники2» с полями «Имя», «Адрес», «Телефон», и сформируйте запросы:

1. Запрос на сочетание данных полей «Родство», «Имя», «Адрес» и «Телефон».
2. Запрос на соединение данных полей «Имя», «Работа», «Адрес» и «Телефон».
3. Запрос на соединение полей «Имя», «Работа», «Адрес» и «Телефон» для одной из профессий.

4. Вычисления и групповые операции в запросах

В ряде случаев при формировании набора данных по запросам на выборку требуется производить определенные вычисления или операции по обработке отбираемых данных. В СУБД такие возможности предоставляются через *вычисляемые поля* и *групповые операции* в запросах над отбираемыми данными.

Вычисляемые поля. В инструкции *SELECT* в списке отбираемых полей добавляется выражение, по которому вычисляется новое поле, и посредством ключевого слова *AS* определяется его имя в формируемом наборе данных. На рисунке 4.1. приведен запрос с вычисляемым полем «ИТОГО».

Сотрудники							
Таб №	Фамилия	Имя	Отчество	Должность	Оклад	Перс_надб	Надб_за_уч.степ
1	Иванов	Иван	Иванович	Начальник	100р.	100р.	50р.
2	Петров	Петр	Петрович	Начальник	80р.	50р.	0р.
3	Сидоров	Сидор	Сидорович	Инженер	40р.	0р.	20р.
4	Егоров	Егор	Егорович	Начальник	80р.	30р.	40р.
5	Кузьмина	Ольга	Игоревна	Секретарь	30р.	150р.	0р.

Ведомость начислений

SELECT Сотрудники.*,
Сотрудники.Оклад+Сотрудники.Перс_надб+Сотрудники.Надб_за_уч_степ AS ИТОГО
FROM Сотрудники;

Таб №	Фамилия	Имя	Отчество	Должность	Оклад	Перс_надб	Надб_за_учстеп	Итого
1	Иванов	Иван	Иванович	Начальник	100р.	100р.	50р.	250р.
2	Петров	Петр	Петрович	Начальник	80р.	50р.	0р.	130р.
3	Сидоров	Сидор	Сидорович	Инженер	40р.	0р.	20р.	60р.
4	Егоров	Егор	Егорович	Начальник	80р.	30р.	40р.	150р.
5	Кузьмина	Ольга	Игоревна	Секретарь	30р.	150р.	0р.	180р.

Рисунок 4.1. - Пример запроса на выборку с вычисляемым полем

Групповые операции. В процессе отбора и обработки данных важное значение имеют группирование данных по значениям какого-либо поля и осуществление операций над сгруппированными записями. Групповые операции осуществляются на основе *SQL*-предложения *GROUP BY* в сочетании со *статистическими функциями SQL*. К числу статистических функций *SQL* относятся:

SUM (выражение) - вычисляет сумму набора значений;

AVG (выражение) – вычисляет среднее арифметическое набора чисел;

Min (выражение) – вычисляет минимальное значение из набора значений;

Max (выражение) – вычисляет максимальное значение из набора значений;

StDev (выражение) – вычисляет среднеквадратичное отклонение значений;

Count (выражение) – вычисляет количество записей, содержащихся в наборе;

Var (выражение) – вычисляет дисперсию по набору значений.

К числу функций, используемых в групповых операциях, относятся также функции **First** (выражение) и **Last** (выражение), вычисляющие (возвращающие), соответственно, первое и последнее значения поля в наборе данных. В выражениях в качестве аргумента допускается использование имен полей таблиц.

Сами групповые вычисления задаются включением в *SQL*-инструкцию *SELECT* вычисляемого поля на основе выражения со статистическими функциями, выполняемыми над наборами данных, формируемыми предложением *GROUP BY*.

На рисунке 4.2. приведен запрос, формирующий итоговые данные по сумме премиальных каждого из сотрудников. Группирование производится по полю «ФИО».

Премирование				
ФИО	Оклад	Премия	Дата	Формулировка
Громов И.И.	100р.	200%	01.01.98	За сложность и напряженность
Громов И.И.	100р.	300%	01.03.98	За сложность и напряженность
Гулов А.А.	99р.	100%	10.01.98	За гуманизм в работе с кадрами
Иванов И.И.	70р.	50%	10.01.98	За творческий подход к делу
Петров П.П.	60р.	50%	10.01.98	За служебное рвение
Попова О.П.	90р.	100%	01.03.98	За сложность и напряженность
Сидоров С.С.	80р.	90%	10.01.98	За принципиальность
Тутова Т.Т.	28р.	90%	01.02.98	За исполнительность
Попова О.П.	90р.	100%	01.01.98	За сложность и напряженность

Общий итог по премиям

SELECT Премирование.ФИО, Sum(Премирование.Оклад*Премирование.Премия) AS
ИТОГО
FROM Премирование
GROUP BY Премирование.ФИО;

ФИО	ИТОГО
Громов И.И.	500р.
Гулов А.А.	99р.
Иванов И.И.	35р.
Петров П.П.	30р.
Попова О.П.	180р.
Сидоров С.С.	72р.
Тутова Т.Т.	25р.

Лабораторная работа № 3. Создайте дополнительно к имеющимся таблицу «Родственники3» с полями «Имя», «Зарплата», «Надбавка», «Премия», вычисляемым полем «Итого», и сформируйте запросы:

1. Запрос на вычисление итоговых доходов на каждого родственника.
2. Запрос на вычисление итоговых сумм дохода только на работающих родственниках.
3. Запрос на вычисление максимального дохода из всех родственников.
4. Запрос на вычисление среднего значения дохода для всех родственников.

5. Подчиненные (сложные) запросы

Источником данных для запросов могут быть результаты выполнения других запросов. Возможны два варианта построения таких запросов.

Первый вариант реализуется через указание в *SQL*-инструкциях в качестве имен таблиц и имен полей имен запросов и полей запросов. Синтаксис таких запросов ничем не отличается от обычных запросов, а его исполнение осуществляется в две фазы. По запуску основного запроса сначала неявно запускается запрос, формирующий источник данных, и по завершению его исполнения запускается основной (внешний) запрос.

Второй вариант реализуется через включение в тело внешней (главной) *SQL*-инструкции внутренней инструкции *SELECT*. При этом результат исполнения внутренней инструкции *SELECT* используется для

формирования условия отбора записей в главном (внешнем) запросе или в качестве выражения для нового вычисляемого поля. Такие запросы называются *подчиненными*.

Использование внутренней инструкции *SELECT* для формирования условий отбора записей во внешнем запросе возможно, например, через предикаты сравнения «для некоторых/для всех» — *ANY, SOME, ALL*.

Конструкция запроса может выглядеть следующим образом:

SELECT ...FROM... WHERE Выражение () [ANY|SOME|ALL] (SELECT...),

где *()* — оператор сравнения.

Как правило, выражение включает поле из списка полей внешней *SQL*-инструкции или функцию от этих полей. Внутренняя инструкция *SELECT* должна возвращать набор данных по одному полю или по вычисляемому полю, при принципиальной сравнимости с выражением во внешней *SQL*-инструкции (по типу данных).

Электромобили			
Марка	Колесная_формула	Вместимость	Запас_хода
«Жук-3»	3x3	15	100
«Звезда1»	2x2	5	150
«МолнияXLC»	2x3	12	300
«Пчела3»	2x2	3	50
«Экология2»	2x1	2	5

Маршруты			
№№	Наименование	Время_года	Расстояние
1	«Романтика»	Весна	20
2	«Культура1»	Осень	50
3	«Экстремальная экзотика»	Зима	100
4	«Экстремально-познавательный»	Зима	150

Электромобили, пригодные для использования на всех маршрутах в зимнее время
SELECT Электромобили.*
FROM Электромобили
WHERE (((Электромобили.Запас_хода)>=All (SELECT Маршруты.Расстояние
FROM Маршруты **WHERE** (((Маршруты.Время_года)="Зима"))));

Марка	Колесная_формула	Вместимость	Запас_хода
«Звезда1»	2x2	5	150
«МолнияXLC»	2x3	12	300

Рисунок 5.1. - Пример подчиненного запроса на основе предиката *ALL*.

Предикаты *ANY* и *SOME* («для некоторых»), являющиеся синонимами, используются для отбора в главной *SQL*-инструкции тех записей, которые удовлетворяют сравнению с какой-либо записью (т.е. хотя бы с одной), из отобранных во внутренней инструкции *SELECT*.

Предикат ALL (для всех) используется для отбора в главном запросе только тех записей, которые удовлетворяют сравнению одновременно со всеми записями, отобранными в подчиненном запросе (рисунке 5.1.).

Лабораторная работа № 4. Используя имеющиеся у Вас таблицы, сформируйте следующие подчиненные запросы:

1. Запрос на вывод данных по самому пожилому родственнику.
2. Запрос на вывод данных по родственникам, живущих в Вашем городе.
3. Запрос на вывод данных по всем работающим и живущим в Вашем городе родственникам.

Индивидуальное итоговое задание

В начале учащийся из таблицы 2 выбирает вариант и для двух таблиц выполняет практическую работу.¹

Содержание задания

1. В режиме Конструктора определить структуры, выбранных Вами двух таблиц вашего варианта задания (см. таблица 1).
2. Заполните созданные таблицы информацией не менее 10 записей в таблице.

Таблица 1. – Состав и структура базы данных

№	Наименование таблицы	Наименование реквизита	Тип реквизита	Примеч.
1.	Покупатели	Код поставщика Наименование поставщика ИНН Директор Адрес Телефон Банковские реквизиты	Текстовый Текстовый Текстовый Текстовый Текстовый Текстовый Текстовый Текстовый	Ключевой
2.	Поставщики	Код покупателя Наименование покупателя ИНН Директор Адрес Телефон Банковские реквизиты	Текстовый Текстовый Текстовый Текстовый Текстовый Текстовый Текстовый Текстовый	Ключевой
3.	Договор	Номер договора Дата заключения договора Код поставщика Код покупателя Дата исполнения договора Сумма договора	Текстовый Дата Текстовый Текстовый Дата Числовой Числовой	Ключевой Ключевой (в зависимости от связываемой таблицы)
4.	Товар	Код товара Наименование товара Модель, марка Единица измерения Производитель	Текстовый Текстовый Текстовый Текстовый Текстовый	Ключевой
5.	Материалы	Код материала Наименование материала Единица измерения Производитель Цена Количество	Текстовый Текстовый Текстовый Текстовый Числовой Числовой	Ключевой
6.	Склад	Код склада Название склада	Текстовый Текстовый	Ключевой

¹ Вариант домашнего задания выбирается по последней цифре номера зачетки студента из таблицы 2.

7.	Остатки	Код склада Дата Код товара Единица измерения Остаток	Текстовый Дата Текстовый Текстовый Числовой	Ключевой Ключевой (в зависимости от связываемой таблицы)
8.	Движение	Код склада Дата Код товара Вид операции Сумма операции	Текстовый Дата Текстовый Текстовый Числовой	Ключевой
9.	Выпуск	Код товара Наименование товара Единица измерения Дата Количество	Текстовый Текстовый Текстовый Дата Числовой	Ключевой
10 .	Цена	Код товара Единица измерения Дата Цена за единицу	Текстовый Текстовый Дата Числовой	Ключевой
11 .	Операции	Код склада Код операции движения Наименование операции	Текстовый Текстовый Текстовый	Ключевой

Таблица 2.

Вариант	Основная таблица	Связанная таблица	Поля связи
0	Товар	Выпуск	Код товара
1	Товар	Цена	Код товара
2	Остатки	Товар	Код товара
3	Движение	Операции	Код склада
4	Остатки	Материалы	Код материала
5	Остатки	Склад	Код склада
6	Покупатели	Договор	Код покупателя
7	Выпуск	Цена	Код товара
8	Поставщики	Договор	Код поставщика
9	Остатки	Движение	Код склада

3. Напишите на языке SQL следующие запросы:

1. Запрос на список данных с повторами, без повторов и первые три.
2. Запрос на сочетание данных четырех полей разных таблиц.
3. Запрос на соединение четырех полей разных таблиц для одного из наименования данных.
4. Запрос на суммирование каких-либо числовых данных из разных таблиц.
5. Запрос на вычисление максимального числового значения из разных таблиц.
6. Запрос на вычисление среднего значения для всех числовых данных.
7. Запрос на вывод данных из всех таблиц по самому минимальному числовому значению.
8. Запрос на вывод всех данных для одного из наименований.